Typed Evidence Licenses for Finite Positive Rule Graphs

Sze Chun Yiu
sze-chun.yiu@fysik.su.se

25 August 2026

Abstract

Boolean dependency graphs can report whether a claim remains reachable after refutation, but not which evidence type licenses the surviving derivation. We define a finite typed semantics for positive conjunctive scientific rule graphs. Independent seeds carry declared licenses; every rule has a license cap; and directly refuted claims receive the empty label. The induced monotone operator has a least fixed point on a finite powerset lattice.

We prove finite convergence, rule-order independence, and a proof-tree characterization: a license reaches a claim exactly when a finite untainted proof tree carries it through every seed and rule cap. Unsupported cycles stay empty, added refutations can only remove licenses, and the retraction contains exactly the claim-license pairs that lose every untainted proof tree. Caps also prevent promotion: post-outcome repairs cannot regain prospective authority, and bounded computation cannot acquire a THEOREM license merely because an untyped conclusion remains reachable.

A deterministic evaluator implements the semantics, while a JSON schema validates document shape. Three bounded cases illustrate forecast falsification, query-specific information falsification, and nonpromotion of a computational frontier. The component is restricted to finite positive rule graphs; it does not model negation, probability, inconsistency, or general scientific judgment.

Keywords: automated reasoning; scientific evidence; provenance; least fixed points; belief revision; executable semantics

1. Introduction

Scientific records mix claims supported by different evidence types: analytic proof, constructive bounds, finite exact computation, prospective prediction, forecast-only evidence, post-outcome repair, and bounded computation awaiting external replay. A counterexample can invalidate one layer while leaving another intact.

An untyped dependency graph can preserve independent derivations, but it can still overpromote a surviving repair. If a post-outcome predictor becomes exact on the observed panel, it should not inherit prospective status merely because the old predictor and the repair share a conclusion string. Similarly, a bounded exact search should not acquire a theorem-grade license by passing through a rule whose conclusion is a theorem-shaped sentence.

We attach evidence licenses to the least fixed point of a positive rule graph. The substrate stays close to positive Datalog and provenance: finite claims, positive conjunctive rules, monotone iteration, and finite proof trees. The scientific addition is explicit nonpromotion. Derivability and evidence license must travel together.

Our contributions are:

1. powerset labels for typed evidence licenses on finite claims;
2. capped conjunctive transfer, which admits a license only when every premise and the rule cap admit it;
3. a proof-tree characterization of the least fixed point;
4. monotone and canonical semantic retraction under direct refutation;
5. a deterministic reusable evaluator and machine-readable schema; and
6. three bounded cases that expose the difference between reachability and licensed scientific use.

2. Claims, licenses, and rules

Let Q be a finite claim set and Λ a finite license set. Example licenses are

$$\mathsf{THEOREM},\quad \mathsf{CONSTRUCTIVE\_BOUND},\quad \mathsf{FINITE\_EXACT},$$
$$\mathsf{PROSPECTIVE},\quad \mathsf{FORECAST\_ONLY},\quad \mathsf{POST\_OUTCOME},$$
$$\mathsf{BOUNDED\_COMPUTATION},\quad \mathsf{EXTERNAL\_REPLAY}.$$

A label is a subset of Λ, ordered by inclusion. Its join is union and its bottom element is the empty set. Each claim q has an independent seed label $\sigma(q) \subseteq \Lambda$.

A positive rule is a triple

$$r = (A \to h, K_r),$$

where $A \subseteq Q$ is a nonempty finite body, $h \in Q$ is the head, and $K_r \subseteq \Lambda$ is the rule cap. For premise labels $(\ell_a)_{a\in A}$, define

$$\tau_r((\ell_a)_{a\in A}) = K_r \cap \bigcap_{a\in A} \ell_a.$$

A license crosses a rule only when every premise carries it and the cap permits it. Rules with empty bodies are represented as independent seeds. Let $R \subseteq Q$ be the directly refuted claims.

3. Least-fixed-point semantics

For a label assignment $x \in (2^\Lambda)^Q$, define

$$F_R(x)_q = \begin{cases} \varnothing, & q \in R, \\ \sigma(q) \cup \displaystyle\bigcup_{r \,|\, \mathrm{head}(r)=q} \tau_r(x|_{\mathrm{body}(r)}), & q \notin R. \end{cases}$$

The operator is monotone. Starting from all-empty labels, iterate to stabilization and denote the result by

$$\mathrm{Lic}(R) = \mathrm{lfp}(F_R).$$

Call an asynchronous evaluation *accumulating* if it initializes every unrefuted claim q with $\sigma(q)$, initializes every refuted claim with the empty label, and thereafter only unions a nonempty rule transfer into an unrefuted rule head. A rule instance is *enabled* for a license λ when every body claim currently carries λ, the rule cap contains λ, the head is unrefuted, and the head does not yet carry λ. It is *fair* if every enabled instance is eventually fired; after each label growth, all rule instances are reconsidered.

Theorem 1 (finite convergence and order independence). Synchronous bottom-up iteration has at most $|Q||\Lambda|$ changing rounds, followed by one stability check. Every fair accumulating rule schedule reaches the same least fixed point.

Proof. With R fixed, labels can only gain licenses. There are at most $|Q||\Lambda|$ claim-license pairs. Under a fair accumulating schedule, each pair having a finite proof tree is eventually added by induction on tree height; no unsupported pair can be added because every firing is an instance of the defining operator. The stable assignment is therefore exactly the least fixed point, independently of schedule. □

4. Typed proof trees

A proof tree for (q, λ) is valid under R when:

1. its root is q, and no node is directly refuted;
2. a leaf a satisfies $\lambda \in \sigma(a)$; and
3. an internal node applies a declared rule $A \to h$ whose cap contains λ, with one child proof tree carrying λ for every antecedent in A.

Theorem 2 (proof-tree equivalence). A license λ belongs to $\mathrm{Lic}(R)_q$ if and only if a finite valid proof tree exists for (q, λ).

Proof. Induction on the first iteration round in which λ enters the label of q constructs a tree. Conversely, induction on tree height shows that each leaf license enters from a seed and crosses every internal rule through its cap. □

Discarding labels and keeping only claims with nonempty labels recovers the ordinary reachability view, but loses the evidence distinction.

5. Cycles and nonpromotion

Rules $a \to b$ and $b \to a$ with no seed labels have the all-empty least fixed point. A cycle supplies a derivation shape, not evidence. If a has a theorem seed and both caps permit THEOREM, that license propagates to b. If either cap excludes it, the license stops at that edge.

Corollary 3 (license conservation). Every license at a conclusion occurs in every leaf seed and every rule cap along at least one finite proof tree.

A repair rule whose cap includes POST_OUTCOME but excludes PROSPECTIVE cannot transmit prospective authority, even when one of its premises happens to carry both. Nonpromotion is therefore enforced by the rule semantics rather than by an informal note attached after evaluation.

6. Retraction under refutation

Theorem 4 (refutation monotonicity). If $R \subseteq R'$, then

$$\mathrm{Lic}(R')_q \subseteq \mathrm{Lic}(R)_q$$

for every claim q.

Proof. For every x, $F_{R'}(x)$ is pointwise contained in $F_R(x)$. Iteration from bottom preserves containment. □

Let $L_{\mathrm{pre}} = \mathrm{Lic}(\varnothing)$ and $L_{\mathrm{post}} = \mathrm{Lic}(R)$. Define

$$\mathrm{Ret}(R) = \{(q, \lambda) : \lambda \in L_{\mathrm{pre}}(q) \setminus L_{\mathrm{post}}(q)\}.$$

Call a post-refutation assignment *proof-supported* when it retains every claim-license pair with a finite untainted proof tree and retains no pair without such a tree. Order retractions by set inclusion.

Theorem 5 (canonical semantic retraction). Relative to the declared seeds, rules, caps, and refutations, L_{post} is the unique proof-supported assignment. Consequently, $\mathrm{Ret}(R)$ removes every and only pair that loses all finite untainted proof trees.

Proof. By Theorem 2, membership in L_{post} is equivalent to the existence of a finite valid proof tree under R. Any proof-supported assignment must therefore equal L_{post}, and its complement relative to L_{pre} must equal $\mathrm{Ret}(R)$. □

This is a semantic retraction relative to the declared system. It makes no claim that the input license policy is the only reasonable scientific policy.

7. Executable semantics

The public JSON Schema first validates document shape and required fields. The reference evaluator then performs semantic validation: claim and rule identifiers must be unique, and every license, rule-body claim, rule head, and refutation must refer to a declared object. After both layers pass, the evaluator performs deterministic bottom-up iteration. Output records the final label of every claim, the number of iterations, and the typed retraction from the unrefuted baseline.

The implementation rejects undeclared claims or licenses, empty rule bodies, duplicate rule identifiers, and malformed caps. Canonical sorting makes equal inputs produce byte-stable semantic outputs. The evaluator is deliberately small enough for line-by-line comparison with F_R; it does not infer caps or scientific policy.

8. Bounded case encodings

The following are synthetic fixtures used to test the typed semantics. Each fixture is self-contained and supplies its own seeds, rules, caps, and refutations.

8.1 Forecast falsification

A synthetic optimization record contains an explicit feasible construction, an independent all-size support theorem, and a compact equality forecast supported by a forecast-labeled seed set. A held-out synthetic result satisfies $C_{\text{exact}} = 10 < 11$, directly refuting the equality and its regime label. The construction and the independent support theorem retain their licenses.

A repaired support-two forecast can carry a theorem-grade license inherited from the independent theorem and a post-outcome license from its construction. Its repair cap excludes PROSPECTIVE, so it cannot be counted as prospective confirmation. The old and repaired panels remain distinct evidence objects.

8.2 Decision survives value and witness falsifiers

A second self-contained fixture seeds a four-index certificate for unary optimality. Its declared counterexamples show that complete pair information does not determine exact improvement or the presence of a triple block, and that all proper interaction marginals do not determine exact value.

Decision, value, and optimizer-structure claims occupy separate nodes. Refuting pair-value sufficiency removes its licenses without touching the independently seeded decision theorem. This is query typing: one representation can carry a theorem license for a decision query and no exact-value license.

8.3 A bounded support frontier does not decide an exact constant

A third self-contained fixture assigns analytic licenses to a width-one generalized-Davenport corridor and a saturation-defect lemma. An exact bounded search excludes a specified obstruction through a declared finite support frontier, but its evidence contract labels the result as bounded computation awaiting external replay.

Any rule from that frontier to a support claim is capped by the same licenses. No proof tree carries THEOREM to either exact candidate value of the generalized Davenport constant or to the associated extremal classification. Repeated implementations by the same research group do not become independent mathematical replication.

9. Relation to prior work

Truth-maintenance systems and belief revision own dependency-directed update [1,2]. Positive Datalog owns least-fixed-point rule semantics [3,4]. Annotated and semiring provenance owns typed query annotations, alternative derivations, recursion, trust annotations, and deletion behavior [5,6]. Work on recursive-query causality relates minimal supports, causes, responsibility, and deletion robustness [7,8].

We claim no generic novelty for fixed points, proof trees, provenance labels, minimal supports, hitting sets, causality, or deletion robustness. The residual is a narrowly scoped scientific component: a finite evidence-license vocabulary, cap-preserving nonpromotion, exact typed retraction, a deterministic evaluator, and bounded cases in which the license distinction changes what may be reported.

10. Reproducibility and limitations

The evaluator is checked against exhaustive fixed-point enumeration on bounded random rule graphs, and unit tests cover unsupported and seeded cycles, cap blocking, alternative derivations, and refutation monotonicity. The proofs carry the general finite authority.

The component handles positive conjunctive rules only. Stratified negation, defaults, probabilistic evidence, and inconsistency are out of scope. The license set and caps are curated policy inputs, not inferred truths. Powerset intersection is one transparent transfer choice. The cases demonstrate machine-checkable behavior in specified records; they are not evidence of broad human-science usability.

11. Conclusion

Reachability alone is not scientific authority. A claim can remain reachable under a weaker license, and a repaired claim can regain exactness without regaining prospective status. The typed least fixed point makes those distinctions explicit and executable.

The component fails closed in two directions: unsupported cycles create no licenses, and underlicensed derivations create no stronger evidence types. Its retraction is canonical relative to the declared inputs: it removes exactly the pairs that lose every valid proof tree. No separate inclusion-minimality claim is made.

Tool-use disclosure

A generative language model assisted manuscript organization, language revision, and submission-package preparation. The listed author remains responsible for the mathematical statements, proofs, references, executable claims, and final text.

Data and code availability

The submission package includes the JSON schema, deterministic Python evaluator, unit tests, and bounded case fixtures required to reproduce every executable claim. These files are distributed in the source archive accompanying this version.

References

1. J. Doyle, "A Truth Maintenance System," *Artificial Intelligence* **12**, 231-272 (1979). DOI: 10.1016/0004-3702(79)90008-0
2. J. P. Martins and S. C. Shapiro, "A Model for Belief Revision," *Artificial Intelligence* **35**, 25-79 (1988). DOI: 10.1016/0004-3702(88)90031-8
3. C. Bourgaux, P. Bourhis, L. Peterfreund, and M. Thomazo, "Revisiting Semiring Provenance for Datalog," in *KR 2022* (2022). DOI: 10.24963/kr.2022/10
4. M. Abo Khamis, H. Q. Ngo, R. Pichler, D. Suciu, and Y. R. Wang, "Convergence of Datalog over (Pre-)Semirings," in *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (PODS 2022), 105-117 (2022). DOI: 10.1145/3517804.3524140
5. T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance Semirings," in *Proceedings of PODS 2007*, 31-40 (2007). DOI: 10.1145/1265530.1265535

6. P. A. Bonatti, A. Hogan, A. Polleres, and L. Sauro, "Robust and Scalable Linked Data Reasoning Incorporating Provenance and Trust Annotations," *Journal of Web Semantics* **9**(2), 165-201 (2011). DOI: 10.1016/j.websem.2011.06.003
7. R. B. Thapa and S. Staab, "Causality and Minimal Supports in Recursive Datalog," arXiv:2607.16443 (2026).
8. R. B. Thapa and S. Staab, "Causal Explanations for Stratified Datalog," arXiv:2608.21141 (2026).